Relatório P6 - Sistemas Operacionais II

Comunicação Segura e Gestão Dinâmica de Grupos

Felipe Gassen, Felipe Murta, Gustavo D'Angelo, Tom Hunt

19/11/2025

1 Requisitos

O objetivo desta etapa do projeto foi estender a API de comunicação para veículos autônomos, focando na segurança da informação e na estruturação lógica da rede em grupos dinâmicos. Os requisitos específicos abordados foram:

- **Autenticação e Integridade (MAC):** Implementação de um *Message Authentication Code* (MAC) em todas as mensagens trocadas (Interesse e Resposta). O algoritmo sugerido foi um XOR simples, utilizando uma chave de sessão distribuída pelo líder do grupo.
- **Grupos Dinâmicos (Quadrantes):** O sistema, que anteriormente operava em uma topologia plana, agora deve suportar a formação de grupos baseados em localização geográfica. Veículos entrando em uma nova área devem se juntar ao grupo local de forma transparente.
- **Monitoramento de Membros:** Os componentes devem ser capazes de identificar quais veículos estão próximos e ativos no grupo, interrompendo o envio de dados para membros que deixaram a região.
- **Gestão de Chaves:** A RSU (Líder do Grupo), que já atuava como mestre de tempo, passa a ser responsável também por gerar e distribuir a chave de sessão para os membros do grupo.

2 Implementação e Arquitetura

Para atender aos requisitos, a arquitetura do sistema foi refatorada para incluir novos cabeçalhos de pacote, handlers de grupo e estruturas de dados na memória compartilhada.

2.1 Segurança e MAC

A segurança foi implementada diretamente na camada de `Protocol`. Definimos uma interface `ICryptoProvider` para permitir a troca futura de algoritmos, com uma implementação inicial `XorCryptoProvider`.

A estrutura do pacote na rede foi alterada para incluir o MAC ao final do payload. O cálculo do MAC abrange os cabeçalhos do pacote e os dados do segmento, garantindo que nem o cabeçalho de roteamento nem os dados foram adulterados.

```
// Estrutura final serializada:
// [ Packet Headers (Localizacao + Portas) | Segmento de Dados | MAC (8
    bytes) ]
```

Listing 1: Estrutura Lógica do Pacote Seguro

No recebimento, o protocolo verifica o MAC antes de qualquer processamento. Se o MAC calculado não coincidir com o recebido, o pacote é silenciosamente descartado.

2.2 Gerenciamento de Grupos e Lista de Presença

A simulação foi dividida em 4 quadrantes geográficos (NORTH, SOUTH, EAST, WEST). Quando um veículo muda de quadrante, a NIC externa detecta a alteração e sinaliza o protocolo. O veículo então envia uma mensagem de `JOIN_REQUEST` em broadcast para a RSU local, que responde com a chave de sessão via unicast (`KEY_DISTRIBUTION`).

Um ponto crucial da implementação foi a **Lista Dinâmica de Entidades Próximas**:

- **Shared Memory:** A `ShmEngine` mantém uma lista de endereços MAC de todos os veículos ativos no quadrante atual. Essa lista é atualizada automaticamente pela NIC sempre que mensagens válidas são recebidas.
- **Monitoramento (TEDS Handler):** O `TEDSHandler` possui uma thread dedicada de limpeza (*cleanup thread*). Periodicamente, ela consulta a lista de entidades próximas na memória compartilhada.
- **Notify Left:** Quando a RSU detecta que um veículo saiu do grupo, ela envia uma mensagem de `NOTIFY_LEFT` em broadcast.
- **Interrupção de Envio:** Se o `TEDSHandler` detecta que um assinante de dados (subscriber) não está mais na lista de próximos (ou recebeu um *notify left*), ele remove esse veículo de sua lista interna e recalcula o intervalo de envio de dados, economizando recursos de processamento e rede.

2.3 Integração com PTP (Existente)

O protocolo PTP, implementado na etapa anterior, foi mantido e integrado à nova estrutura de grupos. A RSU continua sendo a referência de tempo (Mestre), e a sincronização ocorre em paralelo com a gestão de chaves. O desafio nesta etapa foi garantir que as mensagens de sistema do PTP (que não possuem MAC inicial) não fossem bloqueadas pela nova camada de segurança, o que foi resolvido através de uma verificação de tipo de mensagem antes da validação do MAC.

3 Resultados

Os testes foram realizados em um ambiente virtualizado com múltiplas instâncias QEMU.

3.1 Log de Troca de Grupo e Segurança

O log a seguir demonstra a dinâmica de grupos. O veículo muda de quadrante, solicita entrada no novo grupo, recebe a chave e valida mensagens subsequentes.

```
[EXTERNAL NIC] Car moved to quadrant 3.
[EXTERNAL NIC] Sending dirty buffer so protocol knows the car moved.
[PROTOCOL] Handling quadrant change to: 3
[GroupMember] Sending JOIN request to Quadrant 3.
...
[GroupMember] Received KEY_DISTRIBUTION. Stored new session key!
[Protocol] Message received and MAC verified successfully.
```

Listing 2: Troca de Grupo e Distribuicao de Chaves

3.2 Log de Monitoramento de Membros

Aqui observamos a eficácia da lista de presença na memória compartilhada. Quando um veículo para de se comunicar ou sai do grupo, o sensor local interrompe o envio.

```
[GroupMember] Received NotifyLeft packet: Client 02:00...:03 has left!
[TEDS Handler] Removing distant subscriber: 02:00...:03
[TEDS Handler] Subscriber list updated. Recalculating transmission
    period.
```

Listing 3: Monitoramento e Notify Left

3.3 Log de Latência

A seguir apresentamos um registro visual das medições de latência realizadas durante a execução do sistema. O gráfico demonstra o comportamento temporal da comunicação entre veículos e RSU, incluindo variações causadas pela troca de quadrantes, redistribuição de chaves e ajustes dinâmicos de assinantes. A seção General Statistics deve ser ignorada pois o seu propósito de debug não está a prova aqui.

```
## Latency Analysis (Split by Origin)

### SharedMemory
  Average Latency:          0.2426 ms
  Minimum Latency:          0.2143 ms
  Maximum Latency:          0.5529 ms
  Measurements Taken:       43
  P50 (Median) Latency:     0.2236 ms
  P90 Latency:              0.2397 ms
  P95 Latency:              0.3335 ms
  P99 Latency:              0.5230 ms

### External
  Average Latency:          1.2448 ms
  Minimum Latency:          0.2101 ms
  Maximum Latency:          2.6170 ms
  Measurements Taken:       34
  P50 (Median) Latency:     1.1955 ms
  P90 Latency:              2.0753 ms
  P95 Latency:              2.5281 ms
  P99 Latency:              2.6154 ms

## General Statistics
  Total Unique PIDs          : 0
  Components Started         : 0

### TEDS Handler Events
  INTEREST_received        : 6
  RESPONSE_sent            : 10
  RESPONSE_received_Temperature: 3

### Latency Handler Events
  PING_sent                : 90
  ECHO_sent                : 497
```


Figura 1: Testes de Latência

4 Conclusão

A implementação foi bem-sucedida ao adicionar segurança e dinamismo à API. A utilização da memória compartilhada para manter o estado dos vizinhos (*entities nearby*) provou-se uma solução eficiente, permitindo que componentes de aplicação (sensores) reagissem dinamicamente à topologia da rede sem complexidade adicional de comunicação.